BERT Fine-Tuning, GPU Training Efficiency, Memory Optimization, and vLLM Inference

Naga Sai Haritha Gottumukkala
M.S. in Data Analytics, San José State University
DATA 266 -HW4.5 | 1598

Abstract—We fine-tuned BERT on 5,000 TweetEval sentiment examples and studied how GPU choice, training batch size, and memory optimizations change performance. Using rows 598–5597, we made a 3,500/750/750 train/validation/test split. The four-epoch A100 baseline reached 69.47% test accuracy; validation accuracy was highest after epoch 2. At FP32 batch 256, the A100 achieved 247.19 samples/s versus 124.91 on the L4. Both failed at the tested batch 512. On the L4, automatic mixed precision (AMP) reached 353.36 samples/s, while AMP combined with gradient checkpointing and an 8-bit optimizer used 4.043 GiB at batch 256 and supported batch 1024. We also studied vLLM serving and KV-cache estimates. We explicitly distinguish per-request and whole-batch timing and do not claim unmeasured GPU limits or an unperformed A100 Part D rerun.

Index Terms—BERT, GPU throughput, automatic mixed precision, gradient checkpointing, vLLM, memory efficiency.

I. INTRODUCTION

GPU training needs both arithmetic capacity and memory for model weights, gradients, optimizer states, and intermediate activations. Larger batches often increase processed examples per second, but consume additional memory. This assignment measures both effects rather than optimizing only one metric.

Inference creates a similar trade-off: batching may increase output-token throughput while affecting request completion time. We therefore examine fine-tuning and serving with separate metrics. BERT and TweetEval supply a reproducible sentiment-classification workload [1], [2]; vLLM is used for text generation [7].

II. EXPERIMENTAL SETUP AND REPRODUCIBILITY

A. Hardware and software

All experiments ran in Google Colab Pro. The original A100 environment recorded Python 3.13.16, PyTorch 2.11.0+cu130, Transformers 5.20.2, Datasets 4.8.5, Accelerate 1.15.0, and scikit-learn 1.6.1. The serving environment used vLLM 0.32.0. The A100 evidence recorded driver 580.82.07 and a device identifier; separate L4 UUID evidence was not confirmed in the supplied materials.

Table I. Measured GPU environments

GPU	Usable memory	Experiments
A100-SXM4-40GB	39.49 GiB	HW4.a; Part A
NVIDIA L4	22.03 GiB	Parts A, B, D, E

B. Dataset and model

The TweetEval sentiment training source contained 45,615 rows. A pinned dataset builder lacked the expected sentiment configuration, so the fixed sentiment Parquet file was loaded directly. Zero-based source rows 598–5597 inclusive gave exactly 5,000 tweets: 776 negative, 2,292 neutral, and 1,932 positive. A stratified split with seed 1598 yielded 3,500 training, 750 validation, and 750 test rows.

We tokenized with BERT-base-uncased and padded to length 128. The baseline used FP32, AdamW, batch 16, learning rate 2×10^{-5}, weight decay 0.01, linear warmup of 10%, gradient clipping at 1.0, and four epochs. Random generators were seeded; bitwise GPU determinism was not guaranteed.

Table II. Reproducibility summary

Setting	Value
SID4 / SEED / SLICE	1598 / 1598 / 598
Source rows	598–5597 inclusive
Train / validation / test	3500 / 750 / 750
Model	BERT-base-uncased
Max sequence length	128

C. Measurement methodology

Short batch sweeps used a seeded initialization, a warm-up step, and one timed epoch over 3,500 examples. Throughput was the number of examples divided by timed epoch seconds. Peak CUDA allocated memory was measured after resetting the peak statistics. Short sweep memory and full fine-tuning memory are different measurements and are never interchanged.

III. HW4.a: BERT FINE-TUNING AND SENTIMENT

BERT was trained for four epochs and evaluated on validation data after each epoch. The final epoch-4 model was scored on the held-out test set, saved, reloaded and used for custom predictions. Table III shows how training and validation diverged.

Table III. Four-epoch A100 baseline

Ep.	Train loss	Val. loss	Val. acc.	Time (s)
1	0.9124	0.7528	65.87%	19.72
2	0.6196	0.7518	69.07%	18.27
3	0.3898	0.8437	68.00%	18.24
4	0.2325	0.9424	67.47%	18.30

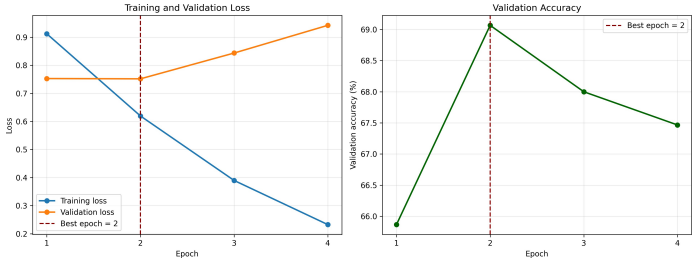


Fig. 1. Training loss declined, while validation accuracy peaked at epoch 2.

Validation accuracy peaked at 69.07% at epoch 2. Two additional epochs, including validation overhead, cost about 39.00 seconds without improving validation performance. Decreasing training loss alongside worsening validation behavior suggests overfitting, but does

not itself prove memorization.

A. Held-out test results and errors

The final epoch-4 model, not an epoch-2 checkpoint, achieved 69.47% test accuracy, test loss 0.9031, macro F1 68.27%, and weighted F1 69.50%. The negative class was weakest by recall (64.96%).

Table IV. Per-class test metrics

Class	Precision	Recall	F1	n
Negative	62.30%	64.96%	63.60%	117
Neutral	68.70%	72.09%	70.35%	344
Positive	73.78%	68.17%	70.86%	289

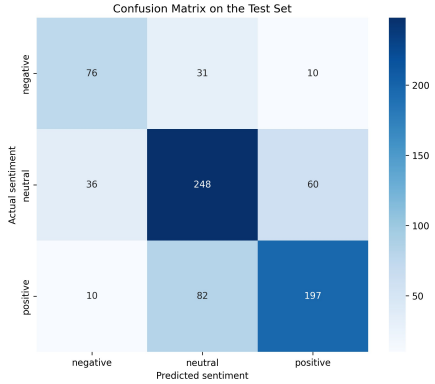


Fig. 2. Test confusion matrix. Thirty-one of 117 negative cases were labeled neutral.

Notebook evidence listed five high-confidence negative-class mistakes (rows 2044, 2097, 5144, 4108, and 2344). They included sarcasm, hopeful expressions mixed with negative sentiment, and factual complaints. Such phrasing made the examples difficult to classify.

IV. HW4.a: HUGGING FACE PIPELINE VS. vLLM

Qwen/Qwen2.5-1.5B-Instruct was served on the A100 using bfloat16. Each original configuration measured 160 requests with up to 64 generated tokens. The original run reported per-request latency; throughput is generated output tokens per second [7], [10], [11].

Table V. Original A100 serving benchmark

Configuration	tok/s	p50 ms	p95 ms
HF pipeline, batch 1	30.31	1330.43	1690.91
vLLM, batch 1	227.67	162.47	231.36
vLLM, batch 8	1353.25	205.82	280.97
vLLM, batch 32	3886.88	310.88	313.20

At batch 1, vLLM delivered approximately 7.5 times the pipeline output-token throughput. Throughput increased with batching, while median per-request latency also rose. These figures describe the original A100 benchmark only.

V. HW4.5 PART A: GPU BATCH-SIZE SCALING

FP32 physical batches 16, 32, 64, 128, 256, and 512 were tested. Both GPUs completed 256 but failed at 512. This is the last successful *tested* size, not the exact absolute memory limit. At batch 256 the A100 reached 247.19 samples/s and the L4 reached 124.91 samples/s.

Table VI. FP32 scaling and allocated CUDA memory

Batch	A100 s/s	L4 s/s	A100 GiB	L4 GiB
16	192.05	98.14	2.479	2.479
32	213.63	114.09	3.741	3.740
64	229.39	118.09	6.156	6.152
128	240.87	124.05	11.046	10.996
256	247.19	124.91	20.716	20.653
512	OOM	OOM	—	—

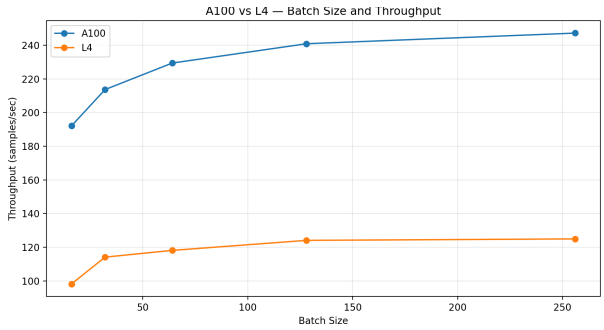


Fig. 3. A100 and L4 training throughput across successful batch sizes.

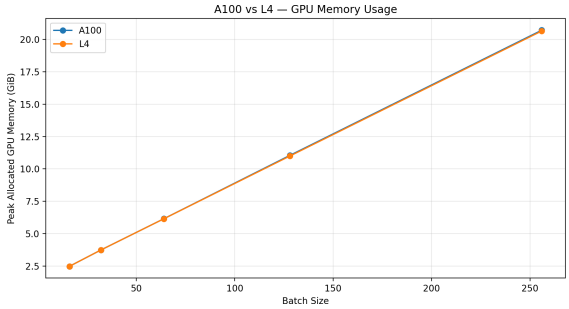


Fig. 4. Peak allocated CUDA memory versus physical batch size.

At batch 256, one sweep epoch took 14.16 seconds on A100 versus 28.02 seconds on L4. The measured available-memory ratio was 39.49/22.03 = 1.79; the largest successful tested batch ratio was 1.00. L4 throughput grew by less than 1% from batch 128 to 256 while memory nearly doubled, showing diminishing returns. Four-epoch multiples of sweep times are estimates, not measured full-run wall clocks.

VI. HW4.5 PART B: MEMORY OPTIMIZATION

A. Automatic mixed precision

AMP applies reduced precision to appropriate operations while retaining higher precision when useful [3]. On L4 at batch 256, throughput increased to 353.36 samples/s versus 124.91 FP32, while peak allocated memory fell from 20.653 to 13.906 GiB. The separate four-epoch batch-16 run took 84.11 seconds and achieved 68.27% test accuracy.

B. Gradient checkpointing

Checkpointing recomputes forward activations during backpropagation, trading computation for memory [4], [5]. At batch 256, it used 6.484 GiB and achieved 90.10 samples/s. It succeeded at batch 1024 and failed at 2048. Its four-epoch run took 194.56 seconds. The saved best checkpoint was re-evaluated without autocast: FP32 test accuracy was 68.53% and test loss 0.8811, matching the earlier FP16-evaluated accuracy.

C. Gradient accumulation

With physical batch 16 and four accumulation steps, effective batch size was 64. Both the matched direct-batch and accumulation experiments processed 3,500 examples in 55 optimizer steps. The direct physical-batch-64 run reached 119.27 samples/s using 11.469 GiB; accumulation reached 113.17 samples/s using 7.002 GiB. The four-epoch accumulation run took 136.24 seconds and achieved 67.47% test accuracy. Different partial-batch handling can affect exact optimization equivalence.

D. Clean 8-bit optimizer rerun

The original 8-bit AdamW test was contaminated by approximately 8.642 GiB of residual CUDA allocations and failed at batch 256. After a clean restart, batch 256 succeeded at 126.96 samples/s using 20.028 GiB; batch 512 failed. Only the clean rerun is used for the final comparison. The four-epoch run took 146.03 seconds with 68.67% test accuracy. Optimizer-state compression saved relatively little at large batch sizes because activations dominated memory [6].

E. Combined optimization and trade-offs

AMP, checkpointing, and 8-bit AdamW together reached 275.87 samples/s at batch 256 using 4.043 GiB. The combined setup supported batch 1024 (277.11 samples/s, 11.747 GiB) but failed at 2048. Its four-epoch batch-16 run took 99.60 seconds and achieved 68.67% test accuracy.

Table VII. L4 optimization comparison (different conditions)

Technique	s/s	GiB	Max batch	Test acc.
FP32 baseline	124.91	20.653	256	—
AMP	353.36	13.906	256	68.27%
Checkpoint	90.10	6.484	1024	68.53%
8-bit AdamW	126.96	20.028	256	68.67%
Combined	275.87	4.043	1024	68.67%
Accum. 16×4	113.17*	7.002*	eff. 64	67.47%

* Accumulation s/s and GiB are from its matched effective-batch-64 comparison, not batch 256. For other rows, throughput and memory are from batch-256 short sweeps; accuracies are from separate four-epoch runs, generally at physical batch 16. Therefore, performance and test accuracy columns were not measured in one identical run.

AMP was fastest at batch 256, while combined optimization used the least memory. On a hypothetical 32 GiB GPU, AMP would be the first technique to try for this workload; combined optimization is useful when memory headroom matters more. A lower-memory method is not automatically the fastest.

VII. HW4.5 PART C: OPTIONAL DISTRIBUTED TRAINING

Part C skipped (optional). No DDP run or two-GPU speedup is claimed.

VIII. HW4.5 PART D: vLLM AND KV CACHE

The later L4 run used Qwen2.5-1.5B-Instruct with vLLM 0.32.0, bfloat16, max_model_len=2048, gpu_memory_utilization=0.75, eager execution, and tensor parallel size 1. Five offline rounds were measured per batch size, with up to 64 generated tokens.

Table VIII. L4 offline vLLM results

Batch	Mean tok/s	p50 batch s	p95 batch s
1	40.45	1.5888	1.5972
8	295.86	1.7136	1.7679
32	1139.74	1.7665	1.8890

Throughput grew 28.2 times between batch 1 and 32; batch-completion p50 rose only about 0.178 seconds. These are percentiles of *whole offline batch durations*, not individually timed request latencies. They are not directly comparable with the HW4.a per-request latency values.

For batch 8, prompt lengths 128, 256, 512, 1024, 1536, and 1984 tokens all succeeded with 16 generated tokens. Estimated active KV-cache demand rose from 0.0273 to 0.4238 GiB; total observed NVIDIA-SMI footprint remained near 16.40–16.44 GiB. The longest successful tested prompt was 1984 tokens, but max_model_len=2048 prevents an absolute hardware context-limit claim. A matching fresh A100 Part D run was not completed.

IX. HW4.5 PART E: EFFICIENCY AND GPU-HOURS

For one GPU, GPU-hours are measured wall-clock seconds divided by 3600. These values describe the full four-epoch batch-16 runs, not extrapolations from the batch-size sweeps.

Table IX. Four-epoch L4 GPU-hours

Method	Seconds	GPU-hours	Test accuracy
AMP	84.11	0.02336	68.27%
Combined	99.60	0.02767	68.67%
Accumulation	136.24	0.03784	67.47%
8-bit AdamW	146.03	0.04056	68.67%
Checkpoint	194.56	0.05405	68.53%

AMP minimized runtime and GPU-hours. Combined optimization required about 18.4% more time than AMP but provided substantially greater batch-256 memory headroom and matched the best test accuracy among these optimized runs. No single configuration dominated all metrics.

X. FAILURE AND ERROR ANALYSIS

The dataset loader initially failed because a pinned revision exposed only a default builder configuration. Loading fixed Parquet directly restored the correct 45,615-row source. A100 and L4 both ran out of memory at batch 512. The first 8-bit benchmark was invalidated by leftover CUDA allocations, and its clean rerun replaced it. Checkpointing deliberately traded throughput for memory.

The original checkpointing evaluation used FP16 autocast although training was FP32; re-evaluation at FP32 preserved 68.53% accuracy. Baseline training continued for two epochs after the validation peak, adding 39 seconds. vLLM installation raised optional dependency warnings, but inference ran. Finally, per-request versus whole-batch latency and the 2048-token engine cap were documented as limitations rather than interpreted as hardware effects.

XI. DISCUSSION AND PRACTICAL RECOMMENDATIONS

The A100 was about twice as fast as the L4 at the successful FP32 batch sizes. A finer batch-size sweep between 256 and 512 might show different actual OOM thresholds. On this BERT workload, AMP was a strong first choice because it improved both speed and memory; combined optimization was preferable when memory capacity was the

priority. Gradient checkpointing is appropriate when a workload otherwise does not fit, while accumulation allows a larger effective update from smaller physical batches.

For serving, larger offline batches improved total token throughput, but an interactive system also needs independent request queue and completion latency. A future experiment should repeat benchmarks, align GPU software versions, collect per-request L4 latency, and rerun A100 Part D with identical prompts and settings.

XII. LIMITATIONS

Results cover one A100 and one L4 runtime without repeated independent trials. CUDA allocated memory excludes some non-PyTorch allocations; NVIDIA-SMI describes a different footprint. Batch maxima are bounded by powers-of-two tests. The latest L4 serving figures measure offline batch durations, and a matching A100 rerun was not performed. KV-cache demand was estimated, not isolated directly, and the 2048-token model setting limited context-length testing. Part C was optional and skipped. Separate L4 UUID and reservation evidence were not verified in the supplied report materials.

XIII. CONCLUSION

The original BERT experiment achieved 69.47% test accuracy and peaked in validation after epoch 2. In batch-scaling trials, the A100 delivered roughly twice the L4 throughput at batch 256, while both failed at the tested batch 512. On L4, AMP was the highest-throughput choice at batch 256; combining AMP, checkpointing, and an 8-bit optimizer used the least memory at that batch while maintaining practical throughput. GPU efficiency must be evaluated across memory, speed, accuracy, and latency rather than by a single metric.

AI-USE DISCLOSURE

An AI assistant helped organize the report .Understand the given task ,concept, and expected results .

REFERENCES

[1] J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” NAACL-HLT, 2019, doi:10.18653/v1/N19-1423.

[2] F. Barbieri et al., “TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification,” Findings of EMNLP, 2020, doi:10.18653/v1/2020.findings-emnlp.148.

[3] PyTorch, “Automatic Mixed Precision examples,” docs.pytorch.org/docs/stable/notes/amp_examples.html.

[4] T. Chen et al., “Training Deep Nets with Sublinear Memory Cost,” arXiv:1604.06174, 2016.

[5] PyTorch, “torch.utils.checkpoint,” docs.pytorch.org/docs/stable/checkpoint.html.

[6] T. Dettmers et al., “8-bit Optimizers via Block-wise Quantization,” ICLR, 2022, arXiv:2110.02861.

[7] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP, 2023, doi:10.1145/3600006.3613165.

[8] NVIDIA, “NVIDIA A100 Tensor Core GPU Data Sheet,” NVIDIA documentation, 2021.

[9] NVIDIA, “NVIDIA L4 GPU Accelerator Specifications,” NVIDIA documentation, 2023.

[10] Hugging Face, “Pipelines,” huggingface.co/docs/transformers/main_classes/pipelines.

[11] Qwen Team, “Qwen2.5-1.5B-Instruct Model Card,” huggingface.co/Qwen/Qwen2.5-1.5B-Instruct.